

FLOW CHART TO CODE
CONVERSION

Q1.

Traverse an array and display elements.

#include <stdio.h>

int main() {

int a[100], n, i;

printf("Enter number of elements:");

scanf("%d", &n);

printf("Enter elements: \n");

for (i=0; i<n; i++)

scanf("%d", &a[i]);

printf("array elements: \n");

for (i=0; i<n; i++)

printf("%d", a[i]);

return 0;

}

Q2

Find Largest element in an array

#include <stdio.h>

int main() {

int a[100], n, i, largest;

printf("Enter number of elements:");

scanf("%d", &n);


```
for (i=0; i<n; i++)
```

```
scanf("%d", &a[i]);
```

```
largest = a[0];
```

```
for (i=1; i<n; i++) {
```

```
    if (a[i] > largest)
```

```
        largest = a[i];
```

```
}
```

```
printf("Largest element = %d", largest);
```

```
return 0;
```

```
}
```

Q2. Find smallest element in an array

```
#include <stdio.h>
```

```
int main () {
```

```
    int a[100], n, i, smallest;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    for (i=0; i<n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    smallest = a[0];
```

```
    for (i=1; i<n; i++) {
```

```
        if (a[i] < smallest)
```

```
            smallest = a[i];
```

```
}
```



```

    printf("smallest element = %d", smallest);
    return 0;
}

```

Q4.

Linear search

```
#include <stdio.h>
```

```
int main () {
```

```
    int a[100], n, key, i, found = 0;
```

```
    printf("Enter number of elements:");
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    printf("Enter key: ");
```

```
    scanf("%d", &key);
```

```
    for(i=0; i<n; i++) {
```

```
        if (a[i] == key) {
```

```
            printf("element found at position %d", i+1);
```

```
            found = 1;
```

```
            break;
```

```
        }
    }
```

```
    if (!found)
```

```
        printf("Element not found");
```

```
    return 0;
```

```
}
```


05.

Binary search

#include <stdio.h>

int main() {

int a[100], n, key, low, high, mid, found = 0;

printf("Enter number of elements: ");

scanf("%d", &n);

printf("Enter sorted elements: \n");

for (int i = 0; i < n; i++)

scanf("%d", &key);

low = 0;

high = n - 1;

while (low <= high) {

mid = (low + high) / 2;

if (a[mid] == key) {

printf("Element found at position %d", mid + 1);

found = 1;

break;

}

else if (a[mid] < key)

low = mid + 1;

else

high = mid - 1;

}

if (!found)

printf("Element not found");

return 0;

}

Q6. Insert element at a specific position

```
#include <stdio.h>
```

```
int main() {
```

```
    int a[100], n, pos, value, i;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    printf("Enter position and value: ");
```

```
    scanf("%d %d", &pos, &value);
```

```
    for (i = n; i >= pos; i--)
```

```
        a[i] = a[i-1];
```

```
    a[pos] = value;
```

```
    n++;
```

```
    printf("Array after insertion: \n");
```

```
    for (i = 0; i < n; i++)
```

```
        printf("%d", a[i]);
```

```
    return;
```

```
}
```


Q7. Bubble sort.

```
#include <stdio.h>
```

```
int main () {
```

```
    int a[100], n, i, j, temp;
```

```
    printf("enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    for (i=0; i<n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    for (i=0; i<n-1; i++) {
```

```
        for (j=0; j<n-i-1; j++) {
```

```
            if (a[j] > a[j+1]) {
```

```
                temp = a[j];
```

```
                a[j] = a[j+1];
```

```
                a[j+1] = temp;
```

```
            }  
        }
```

```
    printf("sorted array:\n");
```

```
    for (i=0; i<n; i++)
```

```
        printf("%d", a[i]);
```

```
    return 0;
```

```
}
```


Q8

Selection Sort.

#include <stdio.h>

int main() {

int a[100], n, i, j, min, temp;

printf("enter number elements: ");

scanf("%d", &n);

for (i = 0; i < n; i++)

scanf("%d", &a[i]);

for (i = 0; i < n - 1; i++) {

min = i;

for (j = i + 1; j < n; j++) {

if (a[j] < a[min])

min = j;

}

temp = a[i];

a[i] = a[min];

a[min] = temp;

}

printf("sorted array");

for (i = 0; i < n; i++)

printf("%d", a[i]);

return 0;

}

Q9

Singly linked list

#include <stdio.h>

#include <stdlib.h>

struct Node {

int data;

struct Node *next;

};

int main() {

struct Node * head = NULL;

* newnode, * temp;

int n, value, i;

scanf("%d", &n);

for (i=0; i<n; i++) {

newnode = (struct Node *) malloc

sizeof (struct Node);

scanf("%d", &value);

newnode->data = value;

newnode->next = NULL;

if (head == NULL) {

head = newnode;

} else {

temp = head;

while (temp->next != NULL)

temp = temp->next;

temp->next = newnode;

}


```
printf("Linked list:");
```

```
temp = head;
```

```
while (temp != NULL) {
```

```
    printf("%d → ", temp->data);
```

```
    temp = temp->next;
```

```
}
```

```
printf("NULL");
```

```
return 0;
```

```
}
```

Q10.

Traverse a singly linked list and display each node.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
int main() {
```

```
    struct node *head, *temp, *newnode;
```

```
    head = malloc(sizeof(struct node));
```

```
    head->data = 10;
```

```
    newnode = malloc(sizeof(struct node));
```

```
    newnode->data = 20;
```



```
head->next = new node /  
new node->next = NULL;
```

```
temp = head;
```

```
while (temp != NULL) {
```

```
    printf("%d ", temp->data);
```

```
    temp = temp->next;
```

```
}
```

```
return 0;
```

```
y-
```